SECURITY

OAuth 2.0 & Authentication Flows

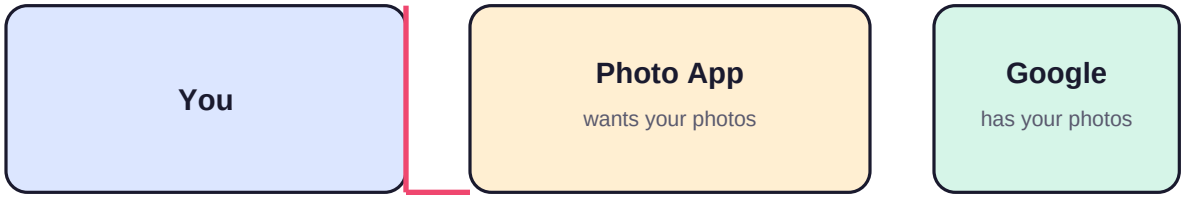
'Log in with Google' —
how delegated auth works without sharing passwords.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Problem OAuth Solves

Apps need access to your stuff — without your password.



✗ Don't give the app your Google password!

✓ Give it a scoped, revocable token instead.

OAuth 2.0 = delegated AUTHORIZATION. OpenID Connect (OIDC) adds authentication on top.

STEP 2/6

Roles in the Dance

Four actors, each with a job.

Resource Owner

You — the human granting access

Client

The app requesting access (photo editor)

Authorization Server

Issues tokens (Google's auth)

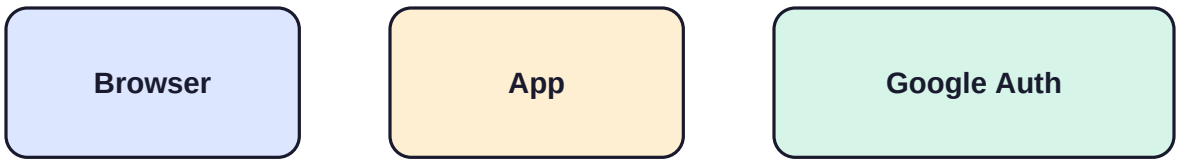
Resource Server

Holds the data (Google Photos API)

STEP 3/6

Authorization Code Flow (the secure one)

The front channel carries a code; the back channel trades it for tokens.



1. → Google: 'user wants to log in'

2. ← login + consent screen

3. → app: authorization CODE

4. app → Google (back channel): code + secret

5. ← access + refresh tokens

Why the code? Tokens never touch the browser URL — the code is single-use, short-lived, and the secret exchange happens server-to-server. PKCE adds proof for public clients.

STEP 4/6

Tokens: Access vs Refresh vs ID

Three tokens, three jobs.

Access token

Short-lived (minutes–hours). 'Bearer' — whoever holds it gets in. Sent per API call.

Refresh token

Long-lived. Gets NEW access tokens without bothering the user. Guard it well.

ID token (OIDC)

JWT with who the user is: sub, email, name. For authentication, not API auth.

`eyJhbGciOiJSUzI1NiJ9.eyJzdWIiOiIxMjM0IiwiZW1haWwiOiJ5b3VAbWFpbCJ9…`

JWT = header.payload.signature — verifiable without a database lookup.

STEP 5/6

Scopes & Consent

Least privilege applies to tokens too.

scope=photos.read

can list and view — NOT delete, NOT read email

- The consent screen shows exactly what's requested — read it before clicking Allow.
- Apps should request minimum scopes; users can revoke anytime from the provider.
- Over-scoped tokens are a breach multiplier — a leaked token can only do what its scopes allow.

STEP 6/6

Hardening Checklist

OAuth done wrong is worse than passwords done right.

- **Always use Authorization Code + PKCE**
  Never implicit flow in new apps.
- **Validate everything**
  Issuer, audience, expiry, signature on every token.
- **Short-lived access tokens**
  Minutes, not days. Rotate refresh tokens.
- **Protect the secret**
  Confidential clients keep secrets server-side only.

STEP ✓I✓

Key Takeaways

OAuth 2.0 — Remember

- OAuth delegates authorization without sharing passwords; OIDC adds authentication.
- Authorization Code + PKCE is the flow to use — code in front, tokens in back.
- Access tokens are short-lived bearers; refresh tokens renew them; ID tokens say who you are.
- Scopes enforce least privilege — request minimum, display honestly, allow revocation.
- Validate issuer/audience/expiry/signature; keep access tokens short-lived.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev